DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1 – CO1

DESIGN, ANALYSE AND VALIDATE AN EFFICIENT SEARCH STRATEGY FOR A LARGE-SCALE STUDENT RECORD SYSTEM

Submitted by: Rajashree S

Department: Computer Science and Engineering

Course: CSA0608 – Design and Analysis of Algorithms

Course Outcome: CO1 – Determine algorithm efficiency using asymptotic notations and mathematical techniques

Abstract

This assignment designs, analyses, implements and compares three distinct strategies for searching a student registration number: iterative linear search, recursive binary search, and hash-table lookup. The analysis includes basic operations, mathematical time functions, best/average/worst cases, asymptotic notation, recurrence formulation and solution, preprocessing costs, experimental testing on datasets from 1,000 to 1,000,000 records, and a final recommendation for a system handling 50,000 searches per hour with daily record additions. The included Python program is reproducible and records execution time and operation counts. Actual timings vary with hardware and should be rerun on the submission computer before final submission.

Problem Statement and Requirements

A university stores student records identified by unique registration numbers. The system must determine whether a requested registration number exists and retrieve its record. The dataset may grow from 1,000 to more than 1,000,000 records. The Day-4 design challenge requires 50,000 searches per hour on 1,000,000 records, while new records are added only once per day.

- Compare three meaningfully different search strategies.
- Analyse mathematical and asymptotic efficiency, including preprocessing.
- Implement and test at 1,000, 10,000, 100,000 and 1,000,000 records.
- Record elapsed time and comparisons/key operations; compare theory with experiment.
- Recommend a design and defend the decision for the changed workload.

Task 1 – Algorithm Design

Approach 1: Iterative Linear Search

Idea: Examine registration numbers from the beginning until the target is found or the list ends. It works on unsorted records and needs no preprocessing.

Pseudocode:

```
LINEAR-SEARCH(A, target)
  for i = 0 to length(A) − 1
    if A[i] = target
      return FOUND
```

return NOT FOUND

Why selected: It is a simple baseline, supports unsorted data, and has no setup cost. Its main limitation is that many records may be checked for each request.

Approach 2: Recursive Binary Search

Idea: Search a sorted list. Compare the target with the middle element. Continue recursively in only the left or right half. Sorting is a preprocessing requirement.

Pseudocode:

BINARY-SEARCH(A, target, low, high)

if low > high: return NOT FOUND

mid = floor((low + high) / 2)

if A[mid] = target: return FOUND

if target < A[mid]: return BINARY-SEARCH(A, target, low, mid − 1)

else: return BINARY-SEARCH(A, target, mid + 1, high)

Why selected: It demonstrates a recursive divide-and-conquer strategy and reduces the search interval by half at each call. It is effective when records are sorted and updates are infrequent.

Approach 3: Hash-Table Search

Idea: Build a hash index that maps each registration number to its record. A query computes/uses the key's hash and checks the corresponding table location. Python's dictionary is used as the hash-table implementation.

Pseudocode:

BUILD-INDEX(records)

index = empty hash table

for each record in records: index[record.registration_number] = record

return index

HASH-SEARCH(index, target)

if target is a key in index: return FOUND and associated record

return NOT FOUND

Why selected: Hash lookup has expected constant-time performance and is suitable for repeated queries. The index requires memory and must be updated when a new record arrives.

Task 2 – Identify the Basic Operation

Approach	Basic operation	Where it occurs	How count grows
Iterative linear search	Equality comparison between current registration number and target	Inside the loop for each examined record	From 1 comparison in the best case to n in the worst case
Recursive binary search	Comparison of target with middle registration number	Once per recursive call	Approximately halves the remaining interval; about $\log_2(n)$ comparisons
Hash-table search	Key lookup / hash-table probe	During dictionary membership or key retrieval	Expected O(1) per lookup under normal hashing assumptions; worst case can degrade with severe collisions

Task 3 – Mathematical Complexity Analysis

3.1 Linear search

Let n be the number of records. If the target is at position k (positions start at 1), the equality test executes k times. Thus $T(n) = c_0 + c_1k$. Best case: k = 1, so $T(n) = \Theta(1)$. If the target is uniformly likely to be at any position and is guaranteed to exist, the expected number of comparisons is $(1 + 2 + \dots + n)/n = n(n+1)/(2n) = (n+1)/2$, giving $\Theta(n)$. If the target is last or absent, n comparisons are needed, giving $\Theta(n)$.

Preprocessing cost: none. Total cost per query remains $\Theta(n)$.

3.2 Recursive binary search

For n sorted records, each call performs a constant number of comparisons and recurses on at most half the input. The recurrence is $T(n) = T(\lfloor n/2 \rfloor) + c$ for $n > 1$, with $T(1) = c$. After j calls, the remaining size

is approximately $n/2^j$. Stop when $n/2^j \leq 1$, which implies $j \geq \log_2 n$. Therefore $T(n) = \Theta(\log n)$ for searching.

Sorting unsorted records costs $\Theta(n \log n)$ with an efficient comparison sort. Total one-time preparation plus q queries is $\Theta(n \log n + q \log n)$. If records are already kept sorted, daily insertion may cost $\Theta(n)$ for an array/list due to shifting; a tree or database index may reduce update costs.

3.3 Hash-table search

Building an index by inserting n keys takes expected $\Theta(n)$ time and $\Theta(n)$ additional space. Under a suitable hash function and controlled load factor, one lookup takes expected $\Theta(1)$, so q lookups take expected $\Theta(q)$. Combined cost is expected $\Theta(n + q)$. In a theoretical worst case where many keys collide, one lookup can take $O(n)$, and q lookups can take $O(qn)$.

3.4 Summary

Approach	**Preprocessing**	**Search time (typical)**	**Extra space**
Linear search	$\Theta(1)$	Best $\Theta(1)$; average/worst $\Theta(n)$	$\Theta(1)$
Recursive binary search	Sort: $\Theta(n \log n)$, if needed	$\Theta(\log n)$	$\Theta(\log n)$ recursion stack
Hash table	Build index: expected $\Theta(n)$	Expected $\Theta(1)$; worst $O(n)$	$\Theta(n)$

Task 4 – Best, Average and Worst-Case Analysis

Algorithm	**Best case**	**Average case**	**Worst case**	**Reason**
Linear search	$\Theta(1)$: target is first	$\Theta(n)$: target position is distributed through list	$\Theta(n)$: target last or absent	May need to inspect each earlier record
Recursive binary search	$\Theta(1)$: middle element matches immediately	$\Theta(\log n)$	$\Theta(\log n)$	Each recursive call halves the search interval

Hash-table lookup	Θ(1): direct bucket/key found	Expected Θ(1)	O(n) under pathological collisions	Performance depends on hash distribution and load factor

The average-case assumptions matter: for linear search, the target is assumed equally likely to occur at any position; for hash search, expected constant time assumes a reasonable hash distribution and implementation.

Task 5 – Asymptotic Analysis

Algorithm	**Upper bound O**	**Lower bound Ω**	**Tight bound Θ (typical)**	**Interpretation at large n**
Linear search	O(n)	Ω(1)	Θ(n) average/worst	Doubling the records can approximately double comparisons
Recursive binary search	O(log n)	Ω(1)	Θ(log n) for full search path	Growing from 1,000 to 1,000,000 adds only about 10 to 20 levels
Hash-table search	Expected O(1); worst O(n)	Ω(1)	Expected Θ(1)	Lookup count usually remains nearly stable as records grow, but relies on hashing and memory

The notation describes growth rather than exact elapsed seconds. Constant factors, cache behaviour, memory allocation, data ordering and the programming language affect measured performance.

Task 6 – Formulate the Recurrence Relation

The recursive approach is binary search. Let T(n) be the maximum time to search n sorted records. One call performs a constant amount of work (middle-index calculation and comparisons) and makes at most one recursive call on about n/2 records. There are not two recursive calls because only one half is searched. The base case occurs when the interval is empty or has one element.

Recurrence: $T(n) = T(\lfloor n/2 \rfloor) + c$ for $n > 1$; $T(1) = c$; $T(0) = c_0$. Here n is the current interval size, the recursive subproblem has size at most $\lfloor n/2 \rfloor$, the number of recursive calls per level is one, and work outside recursion is $\Theta(1)$.

Task 7 – Solve the Recurrence Relation

Use the iteration method because the recurrence has one subproblem of half the size and constant work at each level.

$T(n) = T(n/2) + c$

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

After j expansions: $T(n) = T(n/2^j) + jc$

Stop when $n/2^j \leq 1$, so $2^j \geq n$ and $j \geq \log_2(n)$.

Therefore $T(n) = T(1) + c\lceil \log_2(n) \rceil = \Theta(\log n)$.

This means the number of recursive levels increases very slowly as the record count grows. The result describes the search phase; sorting or building an index must be counted separately.

Task 8 – Implementation and Testing

The attached Python source file implements the three strategies and tests four required dataset sizes. The linear search test target is placed at the final position to show its worst-case comparison count. Binary search uses sorted records. Hash search uses a dictionary. The program also checks an absent target to validate not-found behaviour.

Test cases:

Test case	Target	Expected result	Purpose

TC1	First registration number	Found	Linear best-case correctness
TC2	Middle registration number	Found	Typical successful search
TC3	Last registration number	Found	Linear worst-case comparisons
TC4	Registration number not in dataset	Not found	Validate unsuccessful search
TC5	Dataset sizes 1,000 / 10,000 / 100,000 / 1,000,000	Correct result for all methods	Scalability test

Implementation file: student_search_benchmark.py. Run with Python 3 after installing Matplotlib (`python -m pip install matplotlib`). The program prints elapsed time, comparisons/key operations and preprocessing time; it saves a graph in the current folder.

Task 9 – Experimental Performance

The following table is a measured sample run in the document-generation environment. Elapsed time varies with computer, Python version and system load. Re-run the supplied source code on your own computer and replace the values with your own output before submission.

Records n	**Linear comparisons**	**Linear ms**	**Binary comparisons**	**Binary ms**	**Hash lookup ms**	**Sort ms**	**Hash build ms**
1,000	1,000	0.1016	10	0.0067	0.0004	0.0045	0.0264
10,000	10,000	0.3015	14	0.0046	0.0004	0.0374	0.2967
100,000	100,000	3.4391	17	0.0076	0.0006	0.4343	4.2214
1,000,000	1,000,000	32.5458	20	0.0188	0.0013	9.5966	40.1660

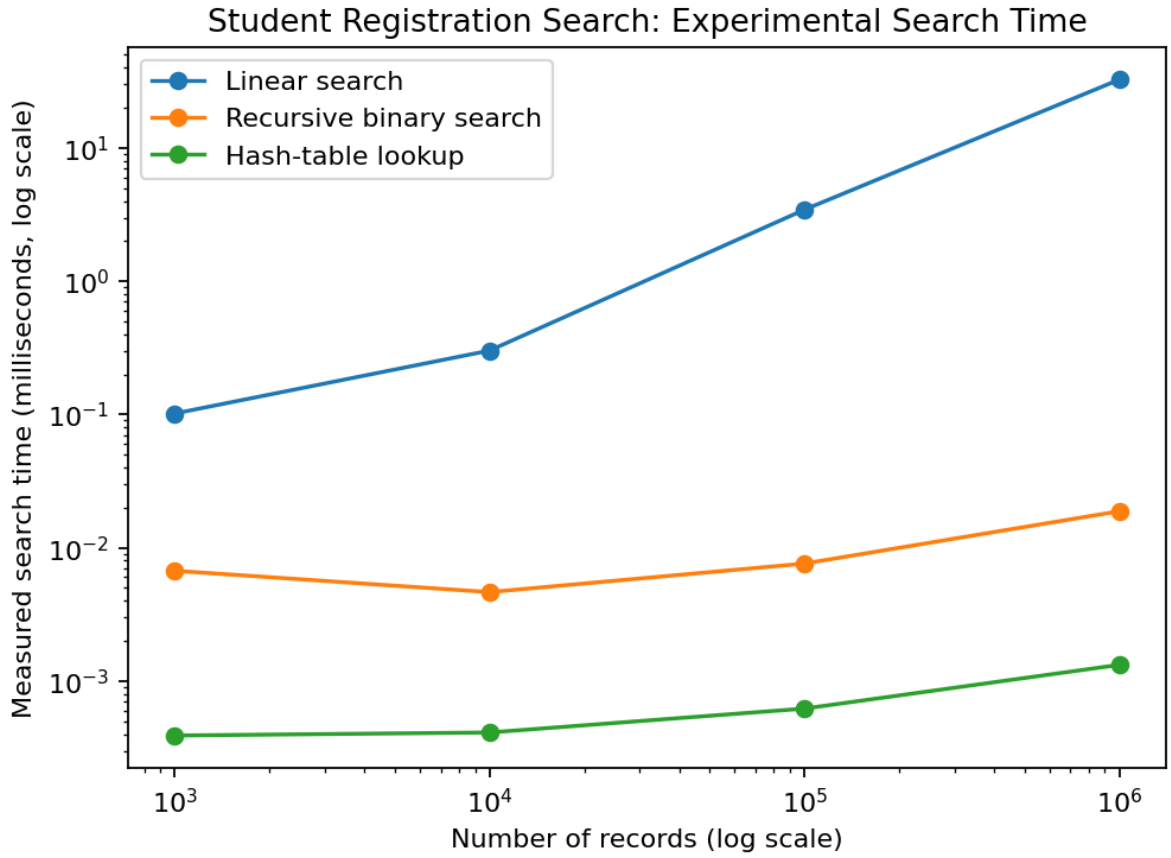


Figure 1. Measured search times from one sample run (logarithmic axes).

Operation counts are more stable than timings: linear search uses n comparisons when the target is last; binary search uses a logarithmic number of comparisons; dictionary lookup is expected constant time. Timings should be interpreted alongside these counts.

Task 10 – Theory versus Experiment

Approach	**Theoretical expectation**	**Expected experimental pattern**	**Possible differences**
Linear search	Θ(n) worst case	Time and comparisons rise approximately in proportion to n	Loop overhead, cache effects, and target position
Recursive binary search	Θ(log n) search; sorting Θ(n log n) separately	Comparisons grow slowly; search remains fast on sorted data	Recursive-call overhead and timer resolution

Hash table	Expected Θ(1) lookup; expected Θ(n) build	Lookup time stays relatively stable while index build grows with n	Memory allocation, hashing, collisions and cache effects

A valid conclusion should use the actual rerun data. If elapsed times are noisy or do not follow the expected curve exactly, operation counts and repeated runs help explain the result. For fair timing, use the same target conditions, repeat each search many times, report the median, and measure preprocessing separately from query time.

Task 11 – Final Recommendation

Recommendation: use a hash-table index for registration-number existence checks and record retrieval when memory is available and the workload is dominated by exact-key searches. Expected lookup time is Θ(1), and daily additions can be handled by inserting each new record into the index. Building the index initially costs expected Θ(n), and the index uses Θ(n) extra space. A production database should use its built-in primary-key or hash/B-tree index rather than a hand-built in-memory structure if persistence, transactions and concurrent access are required.

Recursive binary search is a strong alternative when records must remain sorted or range queries are also important. Its search cost is Θ(log n), but maintaining a sorted array can make insertions expensive. Linear search is suitable only for small lists or infrequent searches because its worst-case query cost is Θ(n).

The final decision must be supported by the experimental results collected on the student's own system; no algorithm should be selected solely because of its Big-O label.

Task 12 – Day-4 Design Challenge

A. Impact Analysis

The revised workload specifies 1,000,000 records and 50,000 search requests per hour, equivalent to about 13.9 searches per second on average. Search frequency is now explicit, so the cost per query accumulates: an O(n) method could examine up to 1,000,000 keys for each unsuccessful or last-position search. New records arrive only once per day, so index maintenance is infrequent relative to searches. This makes a prebuilt lookup index attractive.

B. Evaluate the Original Design

Linear search is not appropriate for this workload: in the worst case it performs 1,000,000 comparisons per request. At 50,000 requests, that could mean up to 50,000,000,000 comparisons per hour. Binary search needs roughly $\log_2(1{,}000{,}000) \approx 20$ comparisons per request, or around one million comparisons across 50,000 requests, after sorting. Hash-table lookup is expected O(1) per request, making it the preferred exact-key strategy when its memory and operational requirements are acceptable.

C. Adaptation or Redesign

Decision: MODIFY the design to use a hash index as the primary registration-number lookup structure. Build or load the index at startup and update it when daily records are added. Validate uniqueness of registration numbers and handle missing keys. In a real system, use a persistent database index and transactions. Keep a sorted index if range queries, ordered reports or prefix/range retrieval are also required.

Complexity after modification: initial index construction is expected $\Theta(n)$; each search is expected $\Theta(1)$; inserting a new record is expected $\Theta(1)$ for the hash index, excluding persistence and resizing overhead. The worst-case lookup can be O(n) under pathological collisions, so use a well-engineered hash implementation and monitor load factor.

D. Evidence-Based Defence

Evidence / constraint	**Implication for design**
1,000,000 records	Linear scan may perform up to 1,000,000 comparisons per query; indexed search is preferable
50,000 searches per hour (~13.9/s average)	Low per-query cost matters across repeated requests
Only one daily update batch	Index construction/maintenance cost is amortized over many searches
Hash lookup expected $\Theta(1)$	Best fit for exact registration-number lookups
Hash index uses $\Theta(n)$ memory	Memory cost must be planned and monitored
Experimental timings	Rerun supplied code on target hardware and attach output screenshots; measured values complement the theoretical argument

Defence: modify to an indexed design using a hash table (or a database's equivalent primary-key index). The recommendation follows from the high query volume, million-record dataset and infrequent updates. It avoids repeated linear scans and avoids the requirement to maintain an array in sorted order. This decision is strongest when queries are exact registration-number lookups; if range or ordered queries are central, a B-tree/sorted index may be preferable.

Conclusion

The analysis shows that linear search is simple but scales poorly, recursive binary search offers logarithmic search on sorted data, and hash-table lookup offers expected constant-time exact-key retrieval at the cost of additional memory and index maintenance. For 50,000 hourly requests against one million student records with daily updates, an indexed hash-based strategy is recommended, with experimental validation on the deployment environment.

Appendix A – Source Code

The complete runnable source code is also supplied as a separate file named student_search_benchmark.py. Use the separate file to execute tests and regenerate the graph. The core functions are shown below.

```
def linear_search(records, target):
    comparisons = 0
    for reg_no in records:
        comparisons += 1
        if reg_no == target:
            return True, comparisons
    return False, comparisons


def recursive_binary_search(sorted_records, target, low=0, high=None, comparisons=None):
    if high is None:
        high = len(sorted_records) - 1
    if comparisons is None:
```

```
        comparisons = [0]
    if low > high:
        return False, comparisons[0]
    mid = (low + high) // 2
    comparisons[0] += 1
    if sorted_records[mid] == target:
        return True, comparisons[0]
    if target < sorted_records[mid]:
        return recursive_binary_search(sorted_records, target, low, mid - 1, comparisons)
    return recursive_binary_search(sorted_records, target, mid + 1, high, comparisons)


def hash_search(index, target):
    return target in index, 1
```

Appendix B – GitHub Repository Structure

```
CO1_Assignment/
├── Task_1_Algorithm_Design/
├── Task_2_Basic_Operation/
├── Task_3_Mathematical_Complexity/
├── Task_4_Case_Analysis/
├── Task_5_Asymptotic_Analysis/
├── Task_6_Recurrence_Formulation/
├── Task_7_Recurrence_Solution/
├── Task_8_Implementation_Testing/Source_Code/
├── Task_9_Experimental_Performance/
├── Task_10_Theory_vs_Experiment/
├── Task_11_Final_Recommendation/
```

```
├── Task_12_Design_Challenge/
├── Report/
│   ├── Source_Code_Screenshots/
│   ├── Output_Screenshots/
│   ├── Analysis/
│   ├── Tables_and_Graphs/
│   └── Documentation/
└── README.md
```

README.md should state the assignment title, Python version, dependencies, algorithms, how to run the program, dataset sizes, where output/graph files are saved, and the repository folder structure.